Магистратура
«Искусственный интеллект»

Инструменты разработки
Лекция 4

Чернов Евгений
НИУ ВШЭ

Bash-скрипты и отладка сервера

Как превратить рутину в надёжный скрипт и найти, почему сервер «лёг»: нагрузка, диск, порты, сервисы и логи

План занятия

1. **Первый скрипт**
   shebang, запуск, кавычки и переменные
2. **Логика скрипта**
   коды возврата, аргументы, [[ ]], if, циклы, функции
3. **Надёжность**
   set -euo pipefail, trap и временные файлы
4. **Отладка и расписание**
   bash -n, bash -x, shellcheck, cron
5. **Диагностика сервера**
   дерево «симптом → команда»: CPU, память, диск, порты
6. **Сервисы и логи**
   systemctl, journalctl и скрипт health-check

Зачем писать скрипты

Скрипт — текстовый файл с командами. Bash читает его сверху вниз и выполняет так же, как если бы вы набирали строки сами. Порядок шагов записан, а не вспоминается по стрелке «вверх».

Рутина руками
10 команд каждый раз,
легко забыть шаг

→

`backup.sh`
записали один раз,
отдали коллеге

→

`cron`
машина повторяет
по расписанию

Правило: что делаете руками больше двух раз — заверните в скрипт. Компьютер не устаёт и не пропускает шаги.

01

РАЗДЕЛ 01

Анатомия и логика скрипта

Первый файл, кавычки и переменные, коды возврата, аргументы, условия, циклы и функции

Первый скрипт: shebang и запуск

терминал

```
$ cat hello.sh
#!/usr/bin/env bash
echo "Привет из $(pwd)"

$ ./hello.sh
bash: ./hello.sh: Permission denied
$ chmod +x hello.sh
$ ./hello.sh
Привет из /home/student
$ bash hello.sh
Привет из /home/student
```

`#!/usr/bin/env bash`
shebang: какой программой читать файл

`chmod +x hello.sh`
разрешить запускать файл как программу

`./hello.sh`
файл из текущей папки; без ./ ищется в PATH

`bash hello.sh`
bash читает файл сам: +x и shebang не нужны

`#!/bin/sh`
на Ubuntu это dash: [[ ]] и массивов там нет

Кавычки и подстановка

Пробел делит строку на слова. Кавычки решают, где слово кончается и подставлять ли значение вместо $. Главная привычка — брать переменные в двойные кавычки.

терминал

```
$ dir="devtools bash"
$ mkdir "$dir"          # одна папка
$ cd $dir
bash: cd: too many arguments
$ cd "$dir"             # сработало

$ name=port
$ echo "${name}_log | $name_log |"
port_log |  |
```

Запись	Что делает bash
`"$var"`	подставит значение одним словом
`'$var'`	оставит текст как есть, без подстановки
`${var}_x`	скобки отделяют имя от хвоста
`$(cmd)`	подставит вывод команды
`$var`	без кавычек: режет по пробелам

Переменные: кому они видны

Скрипт — отдельный дочерний процесс. Он видит только то, что родитель отдал ему через окружение, и не читает ваш ~/.bashrc.

терминал

```
$ cat envdemo.sh
#!/usr/bin/env bash
echo "BOBA=$BOBA"
$ BOBA=123
$ ./envdemo.sh
BOBA=
$ export BOBA
$ ./envdemo.sh
BOBA=123
```

name=value
без пробелов вокруг =; живёт в этом shell

export NAME
копия уходит дочерним процессам

source dev.sh
выполнить файл в текущем shell: cd и export останутся

~/.bashrc
скрипт его не читает: алиасов там нет

Код возврата: 0 — успех

Текст ошибки уходит в stderr, а об успехе команда сообщает числом. 0 — всё вышло, любое другое число — неуспех. Вся логика скриптов строится на этом числе.

терминал

```
$ grep root /etc/passwd; echo $?
root:x:0:0:root:/root:/bin/bash
0
$ ls /nope; echo $?
ls: cannot access '/nope': No such file or directory
2
# число выбирает программа: GNU ls — 2, ls в macOS — 1
$ mkdir -p logs && cd logs
$ cp a.txt b.txt || echo "копия не вышла"
```

`$?`
код прошлой команды; читать сразу: code=$?

`exit 1`
закончить скрипт с неуспехом

`a && b`
b — только если a успешна

`a || b`
b — только если a упала

Аргументы скрипта

Имя файла или папки не зашивают в текст скрипта, а передают при запуске. Сначала проверяем, сколько аргументов пришло, и только потом читаем $1.

Запись	Что это
`$0`	имя скрипта, как его вызвали
`$1, $2 …`	первый, второй аргумент
`$#`	сколько аргументов передали
`"$@"`	все аргументы, каждый — целиком
`${1:-dflt}`	$1, а если его нет — dflt

args.sh

```
#!/usr/bin/env bash
set -euo pipefail
if [[ $# -lt 1 ]]; then
  echo "usage: $0 ФАЙЛ..." >&2
  exit 1
fi
for name in "$@"; do
  echo "аргумент: $name"
done
$ ./args.sh "мой файл.txt" b
аргумент: мой файл.txt
аргумент: b
```

Проверки: команда [[...]]

[[...]] — команда-вопрос: отвечает кодом 0 («да») или 1 («нет»). Пробелы после [[и перед]] обязательны.

Проверка	Вопрос
`-f путь`	это обычный файл?
`-d путь`	это каталог?
`-e путь`	существует хоть что-то?
`-z "$s"  -n "$s"`	строка пустая / непустая?
`-eq -ne -lt -gt`	сравнение чисел: = ≠ < >
`==  !=`	сравнение строк

! `[[ '02' == '2' ]]` → `1`
== сравнивает текст; [[02 -eq 2]] → 0

! `[[ 100 > 90 ]]` → `1`
> тоже строки: «1» раньше «9». Числа — -gt

! `[[ $f == a* ]]`
справа без кавычек — шаблон; 'a*' — буквально

! `[ -n $x ]` → `0 при пустом x`
в старых [ ] без кавычек проверка врёт

Ветвления и циклы

if / elif / else и for

```
if [[ -f config.txt ]]; then
  echo "конфиг есть"
elif [[ -d config ]]; then
  echo "это каталог"
else
  echo "нет конфига" >&2
fi
for i in 1 2 3; do
  echo "шаг $i"
done
```

while и case

```
n=0
while [[ $n -lt 3 ]]; do
  n=$((n + 1))
done
case "${1:-}" in
  start) echo "запускаю" ;;
  stop)  echo "останавливаю" ;;
  *)     echo "usage: start|stop" >&2
         exit 1 ;;
esac
```

if смотрит на код возврата команды: 0 — ветка then, иначе — else. Вместо [[ ]] годится любая команда, например `grep -q err app.log`

Функции: повтор без копипасты

fn.sh

```
#!/usr/bin/env bash
set -euo pipefail

die() {
  echo "ошибка: $1" >&2
  exit 1
}
has_file() {
  local path="$1"
  [[ -f "$path" ]]
}

has_file app.py || die "нет app.py"
echo "app.py на месте"
```

имя() { … }
объявить; вызывать как команду: die "текст"

$1 внутри функции
аргумент функции, а не скрипта

local
переменная только на этот вызов

return и exit
return — выйти из функции, exit — из скрипта

код функции
код последней команды: has_file годится для if

02

РАЗДЕЛ 02

Надёжность, отладка и расписание

Строгий режим, уборка через trap, трассировка bash -x и запуск по cron

Строгий режим: set -euo pipefail

без строгого режима

```
#!/usr/bin/env bash
cd /data/tmp      # папки нет — ошибка…
rm -rf ./*        # …и удалим файлы ЗДЕСЬ
```

со строгим режимом

```
#!/usr/bin/env bash
set -euo pipefail
cd /data/tmp
cd: /data/tmp: No such file or directory
# остановка с кодом 1 — до rm не дошли
```

`set -e`
команда вернула не 0 — скрипт остановлен

`set -u`
незаданная переменная — ошибка, а не пустота

`-o pipefail`
конвейер a | b падает, если упал любой участник

`if, &&, ||`
неуспех в условии — не авария: else сработает

backup.sh: строгий режим и trap

backup.sh

```
#!/usr/bin/env bash
set -euo pipefail
if [[ $# -ne 1 ]]; then
  echo "usage: $0 <папка>" >&2
  exit 1
fi
src="$1"
dst="$HOME/backups"
tmp="$(mktemp)"
trap 'rm -f "$tmp"' EXIT
tar -czf "$tmp" -C "$src" .
mkdir -p "$dst"
cp "$tmp" "$dst/$(basename "$src")-$(date +%F).tgz"
echo "готово: $dst"
```

1. **`$# -ne 1`** — нет аргумента: usage и exit 1
2. **`mktemp`** — временный файл с новым именем
3. **`trap … EXIT`** — rm при любом выходе, даже при ошибке
4. **`tar -czf`** — сжать папку во временный файл
5. **`$(date +%F)`** — дата в имени архива

Сначала $#, потом $1: иначе set -u оборвёт скрипт с «$1: unbound variable».

Отладка: bash -n и bash -x

Не запускается — ищем ошибку синтаксиса. Запускается, но делает не то — смотрим, какие команды bash выполнил на самом деле, уже после подстановки переменных.

copy.sh — найдите опечатку

```
#!/usr/bin/env bash
src="$1"
mkdir -p /tmp/backup
for f in "$scr"/*.txt; do
  cp "$f" /tmp/backup/
done
```

трассировка

```
$ bash -x copy.sh data
+ src=data
+ mkdir -p /tmp/backup
+ for f in "$scr"/*.txt
+ cp '/*.txt' /tmp/backup/
cp: cannot stat '/*.txt': No such file…
```

bash -n file.sh
только синтаксис: забытый fi, кавычка

bash -x file.sh
печатает каждую команду с «+»

set -x … set +x
трассировка одного участка

shellcheck
линтер: кавычки, опечатки, грабли

cron: запуск по расписанию

cron — планировщик задач. Строка crontab — пять полей времени, затем команда. Звёздочка значит «любое значение».

0	3	*	*	1	/home/user/backup.sh
минута 0–59	**час** 0–23	**день месяца** 1–31	**месяц** 1–12	**день недели** 0–7, вс = 0 или 7	**команда** лучше с полным путём

Строка	Когда
`*/5 * * * *`	каждые 5 минут
`0 3 * * 1`	по понедельникам в 03:00
`30 9 * * 1-5`	в 9:30 по будням

терминал

```
$ crontab -e      # редактировать
$ crontab -l      # показать задачи
# вывод — в лог, иначе он теряется:
0 3 * * 1 /home/user/backup.sh \
  >> /tmp/backup.log 2>&1
```

03

РАЗДЕЛ 03

Диагностика сервера

Симптом → команда → что искать в выводе: нагрузка, диск, порты, сервисы и логи

Дерево диагностики «сервер лёг»

Не гадайте: идите от симптома к команде и смотрите в выводе конкретную вещь.

СИМПТОМ	КОМАНДА	ЧТО ИСКАТЬ В ВЫВОДЕ
Сайт не открывается	`ss -tlnp · curl -I`	слушает ли кто-то порт, что отвечает
Всё тормозит	`uptime · top`	load average против nproc, %CPU
No space left on device	`df -h · du -sh`	Use% под 100%, кто съел место
Сервис не работает	`systemctl status`	active или failed, код выхода
Непонятно почему	`journalctl -u svc`	последние ошибки этого сервиса

Нагрузка: CPU и память

Сервер тормозит — первым делом uptime и top: какова очередь к процессору и кто ест CPU и память.

терминал · колонки top сокращены

```
$ uptime
… load average: 4.51, 3.98, 2.10
$ nproc
2
$ top
%Cpu(s): 91.2 us,  5.1 sy,  3.7 id,  0.0 wa …
MiB Mem :  7861.0 total,   210.4 free …
    PID USER      %CPU  %MEM COMMAND
   3120 student   88.5  12.4 python3
   2044 postgres  10.2  30.1 postgres
```

load average
очередь за 1, 5, 15 мин; больше nproc — перегруз

us · sy · id · wa
программы · ядро · простой · ждём диск

free -h
память: смотрите available, а не free

htop
то же нагляднее: цвета, дерево, мышь

kill PID
остановить процесс, когда виновник найден

Диск и порты: df, du, ss

Две частые причины «всё сломалось»: кончилось место на диске или нужный порт никто не слушает.

диск

```
$ df -h /
Filesystem  Size  Used Avail Use% Mounted on
/dev/sda1    40G   38G  512M  99% /
$ sudo du -sh /var/log/* | sort -h | tail -2
850M    /var/log/nginx
1.9G    /var/log/journal
```

порты · колонки сокращены

```
$ sudo ss -tlnp
State  Local Address:Port   Process
LISTEN 0.0.0.0:80           "nginx"
LISTEN 0.0.0.0:22           "sshd"
LISTEN 127.0.0.1:5432       "postgres"
$ curl -I http://localhost
HTTP/1.1 200 OK
```

df — сколько места на разделе, **du** — кто его занял, **ss -tlnp** — кто слушает TCP-порт. 127.0.0.1 — только изнутри сервера, 0.0.0.0 — для всех.

Сервисы и логи: systemctl, journalctl

терминал

```
$ systemctl status nginx
● nginx.service - A high performance web server…
     Active: failed (Result: exit-code)
$ journalctl -u nginx -n 20 --no-pager
… [emerg] unexpected "}" in /etc/nginx/nginx.conf:42
$ sudo nginx -t          # проверить конфиг
$ sudo systemctl restart nginx
$ tail -f /var/log/nginx/error.log
```

status · restart
состояние; start · stop · enable

journalctl -u svc -n 50
последние 50 строк сервиса

journalctl -p err -b
ошибки с последней загрузки

journalctl -f · tail -f
следить за логом онлайн

status
что с сервисом → **journalctl**
почему упал → **правка**
конфиг, место, порт → **restart + проверка**
status, curl

health-check.sh: проверка по расписанию

health-check.sh

```
#!/usr/bin/env bash
set -euo pipefail
# 1. диск занят больше чем на 90%?
use=$(df --output=pcent / | tail -1 | tr -dc '0-9')
if [[ $use -gt 90 ]]; then
  echo "ALERT: диск / занят на ${use}%" >&2
fi
# 2. nginx жив? нет — перезапустить
if ! systemctl is-active --quiet nginx; then
  systemctl restart nginx
fi
```

`df --output=pcent`
только колонка Use%; tr оставит цифры

`[[ $use -gt 90 ]]`
числа сравниваем -gt, а не >

`is-active --quiet`
молчит и отвечает кодом возврата

`if ! …`
«!» переворачивает код: не активен → restart

sudo crontab -e: `*/5 * * * * /usr/local/bin/health-check.sh >> /var/log/health.log 2>&1`

Грабли новичка — и как их обойти

`name = 5`
пробелы вокруг = — bash ищет команду name. Верно: name=5

`rm $file`
имя с пробелом развалится на два слова. Пишите "$file"

`[[ $n > 90 ]]`
> сравнивает строки: 100 «меньше» 90. Для чисел -gt

`env: 'bash\r': No such file`
переводы строк Windows (CRLF): sed -i 's/\r$//' file.sh

`Permission denied`
нет права на запуск: chmod +x или bash file.sh

`в cron не работает`
короткий PATH, нет вашего окружения, % надо писать как \%

ИТОГИ ЛЕКЦИИ

Что важно унести

1 **Скрипт — команды в файле**
shebang, chmod +x, "$var" — всегда в кавычках

2 **Логика на кодах возврата**
0 — успех; if, &&, || и [[ ]] читают это число

3 **Надёжность и отладка**
set -euo pipefail, trap, bash -x, cron с полными путями

4 **Диагностика по дереву**
симптом → top · df · ss · systemctl · journalctl

Дальше в курсе — **Сеть**: SSH, HTTP, WebSocket, gRPC.


ВЫСШАЯ • ШКОЛА • ЭКОНОМИКИ •